



دانشگاه صنعتی ارومیه
وزارت علوم، تحقیقات و فناوری

عنوان پروژه :

شناسایی تقلب در کدهای دانشجویی

نگارش:

احسان حمزه خانی

۹۹۱۱۵۲۱۱۴

استاد راهنما:

دکتر وحید نصراللهی

پروژه کارشناسی رشته مهندسی کامپیوتر-هوش مصنوعی



تقديم به:

پدر و مادر عزيزم كه در اين مسير هموار پشتوانه استواري
براي من بوده اند....

با تشکر از:

جناب آقای دکتر وحید نصراللهی که همواره با زحمات بی
منت همیشه پاسخگو و دلسوز من در این مسیر بوده اند

چکیده.....	۶
فصل اول - مقدمه.....	۷
مقدمه.....	۷
تعاریف کلی.....	۸
فصل دوم مروری بر کار های مشابه و مبانی تحقیق.....	۹
مقدمه.....	۹
فصل سوم روش انجام پروژه و پیاده سازی.....	۱۰
مقدمه.....	۱۰
توضیحات فنی.....	۱۰
ساختار پروژه.....	۱۲
توضیحات نحوه عملکرد سیستم.....	۱۴
نحوه اجرای پروژه.....	۲۰
تصاویری از محیط پروژه.....	۲۱
نمودار ها.....	۲۲
خطاهای احتمالی و اصلاح آن.....	۳۰
مطالعه منابع.....	۲۶
مراجع.....	۳۰
ارائه پیشنهادات.....	۳۲

چکیده:

با توجه به گسترش روزافزون آموزش‌های آنلاین و پروژه‌های برنامه‌نویسی، یکی از چالش‌های مهم در ارزیابی دانشجویان، تشخیص تقلب و تشابه در کدهای ارسالی است. پروژه حاضر به منظور ارائه‌ی سیستمی طراحی شده است که بتواند با استفاده از الگوریتم‌های پیشرفته‌ی یادگیری ماشین و ابزارهای تحلیل کد، به شناسایی دقیق موارد تقلب در پروژه‌های برنامه‌نویسی بپردازد. این سیستم از چندین رویکرد مختلف برای تشخیص شباهت‌ها بهره می‌برد. از جمله، مقایسه‌ی درخت انتزاعی نحوی (AST) که ساختارهای عملکردی کد را تحلیل می‌کند، شباهت توکن‌ها که به تجزیه و تحلیل المان‌های متنی می‌پردازد، و شباهت لُنشتاین که تغییرات جزئی در کدها را مورد بررسی قرار می‌دهد.

برای بهبود دقت تشخیص، همچنین ویژگی‌های اضافی از جمله تعداد توابع، تعداد متغیرها، نسبت نظرات به کد، و پیچیدگی سایکلوماتیک نیز از هر پروژه استخراج شده است. این ویژگی‌ها به همراه نتایج مقایسه‌ای، از طریق مدل‌های یادگیری ماشین مانند XGBoost پردازش شده‌اند تا بتوانند با دقت بالایی تقلب در کدها را پیش‌بینی کنند.

نتایج ارزیابی‌های انجام شده نشان می‌دهد که مدل پیشنهادی، با دقت بالا قادر به تشخیص موارد تقلب بوده و می‌تواند در شناسایی شباهت‌های ساختاری و معنایی بین کدهای مختلف موفق عمل کند. این پروژه نه تنها به کاهش تقلب در پروژه‌های دانشجویی کمک می‌کند، بلکه به بهبود کیفیت ارزیابی‌ها و افزایش عدالت در فرآیند آموزش منجر خواهد شد. کاربردهای بالقوه این سیستم می‌تواند در سایر حوزه‌های تحلیل کد و ارزیابی عملکرد برنامه‌نویسان نیز مفید باشد.

کلید واژه ها : تشخیص تقلب برنامه نویسی، یادگیری ماشین، شباهت کد، درخت AST،

پیچیدگی سایکلوماتیک، الگوریتم XGBOOST

فصل اول

مقدمه:

با رشد فزاینده برنامه‌های آموزشی و دوره‌های آنلاین در حوزه علوم کامپیوتر، مشکل تقلب در تکالیف برنامه‌نویسی به چالشی جدی تبدیل شده است. در این راستا، تشخیص تقلب در کدهای دانشجویان اهمیت بسیاری یافته است. روش‌های سنتی برای شناسایی شباهت کدها غالباً ناکارآمد هستند و به دلیل پیچیدگی بالای ساختارهای برنامه‌نویسی، ممکن است قادر به شناسایی تغییرات جزئی در کدهایی که از یکدیگر کپی شده‌اند نباشند.

در این پروژه، سیستم پیشرفته‌ای برای تشخیص تقلب در کدهای برنامه‌نویسی توسعه داده شده است که از ترکیب روش‌های تحلیل شباهت متنی، تجزیه توکن‌ها، مقایسه درخت انتزاعی نحوی (AST) و محاسبه پیچیدگی سایکلوماتیک استفاده می‌کند. علاوه بر این، از یک مدل یادگیری ماشین، به‌ویژه الگوریتم XGBoost، جهت پیش‌بینی وقوع تقلب بر اساس ویژگی‌های استخراج‌شده از کدها بهره‌برده شده است. این سیستم قادر است با دقت بالا شباهت کدهای برنامه‌نویسی را شناسایی کرده و موارد مشکوک به تقلب را به صورت دقیق گزارش دهد.

تعاریف کلی:

1. **شباهت کد: (Code Similarity)**
شباهت کد به میزان هم‌پوشانی یا مشابهت بین دو برنامه یا قطعه کد اشاره دارد. این شباهت ممکن است از نظر ساختاری، نحوی (Syntax)، یا عملکردی باشد.
2. **درخت انتزاعی نحوی: (AST - Abstract Syntax Tree)**
درخت انتزاعی نحوی ساختاری است که نحوه تجزیه و تحلیل کد را از منظر نحو و ساختار نشان می‌دهد. AST در مقایسه ساختاری کدها برای تشخیص تقلب نقش مهمی دارد.
3. **پیچیدگی سایکلوماتیک: (Cyclomatic Complexity)**
پیچیدگی سایکلوماتیک معیاری برای اندازه‌گیری میزان پیچیدگی یک برنامه از نظر تعداد مسیرهای مستقل اجرایی است. این معیار به شناسایی کدهای پیچیده و ارزیابی احتمال شباهت آن‌ها کمک می‌کند.
4. **توکن: (Token)**
توکن‌ها اجزای کوچک و مستقل یک برنامه هستند که شامل کلمات کلیدی، متغیرها، عملگرها و علائم هستند. مقایسه توکن‌ها یکی از روش‌های شناسایی شباهت کدهاست.
5. **الگوریتم لون‌اشتاین: (Levenshtein Distance)**
الگوریتمی برای اندازه‌گیری فاصله یا تفاوت بین دو رشته از کاراکترهاست. این الگوریتم اغلب برای مقایسه شباهت‌های متنی بین کدها به کار می‌رود.
6. **مدل یادگیری ماشین: (Machine Learning Model)**
یادگیری ماشین به فرایندی اشاره دارد که طی آن یک مدل با استفاده از داده‌های گذشته، الگوها را شناسایی کرده و قادر به پیش‌بینی رفتارهای آینده، از جمله شناسایی تقلب در کد، می‌شود.

فصل دوم مروری بر کارهای مشابه و مبانی تحقیق

مقدمه

در این فصل، به بررسی مشخصات عمومی گزارش پرداخته می‌شود. گزارش‌های مربوط به تحلیل تشابه کد و شناسایی

تقلب به عنوان ابزاری مؤثر در محیط‌های آموزشی و دانشگاهی شناخته می‌شوند. این گزارش‌ها شامل ارزیابی دقیق

شباهت‌های موجود بین کدهای برنامه‌نویسی دانشجویان هستند تا با استفاده از الگوریتم‌های تحلیل متنی و ساختاری،

موارد مشکوک به تقلب شناسایی شوند. همچنین، استفاده از مدل‌های یادگیری ماشین در این سیستم‌ها، دقت و قابلیت

تشخیص تقلب را افزایش داده است.

در این بخش از گزارش، ساختار کلی و اصول پایه‌ای که برای تنظیم و تهیه گزارش‌های تشخیص تقلب استفاده شده است،

توضیح داده می‌شود. این مشخصات شامل روش‌های تحلیل، متریک‌های استفاده شده برای ارزیابی شباهت، و فرایندهای

پردازش و ارائه نتایج به صورت شفاف و قابل درک است.

فصل سوم روش انجام پروژه و پیاده سازی

مقدمه

در این فصل، به تشریح روش‌های به کار گرفته شده برای انجام پروژه و پیاده‌سازی سامانه تشخیص تقلب و شباهت در کدهای برنامه‌نویسی پرداخته می‌شود. برای دستیابی به اهداف پروژه، از الگوریتم‌های مختلفی در حوزه‌های یادگیری ماشین، تحلیل شباهت متنی و ساختاری، و استخراج ویژگی‌های مرتبط با کدها استفاده شده است. این روش‌ها با توجه به ماهیت مسئله و داده‌های موجود انتخاب شده‌اند تا دقت و کارایی سیستم بهینه‌سازی شود.

در ابتدا، به بررسی مراحل جمع‌آوری و آماده‌سازی داده‌ها می‌پردازیم که شامل انتخاب و پردازش کدهای نمونه، برچسب‌گذاری داده‌ها و استخراج ویژگی‌های اصلی کدها می‌باشد. سپس الگوریتم‌های مختلفی که برای تشخیص شباهت میان کدها استفاده شده‌اند، مانند مقایسه ساختارهای نحوی (AST)، تحلیل توکن‌ها و شباهت‌های متنی، معرفی و توضیح داده خواهند شد.

همچنین، پیاده‌سازی مدل‌های یادگیری ماشین برای پیش‌بینی تقلب در کدها و نحوه تنظیم پارامترها، آموزش و ارزیابی مدل‌ها به تفصیل شرح داده خواهد شد. در نهایت، به ابزارها و کتابخانه‌های مورد استفاده در این پروژه اشاره می‌شود که نقش اساسی در پیاده‌سازی سیستم تشخیص تقلب ایفا می‌کنند.

توضیحات فنی:

در این پروژه برای پیاده‌سازی سیستم تشخیص تقلب و شباهت در کدهای برنامه‌نویسی، از ترکیب چندین تکنولوژی و ابزارهای پیشرفته استفاده شده است. زبان اصلی برنامه‌نویسی برای توسعه این سامانه پایتون است. دلیل انتخاب پایتون، قدرت و انعطاف‌پذیری بالای آن در انجام پردازش‌های متنی و استفاده از کتابخانه‌های یادگیری ماشین است.

تکنولوژی‌ها و زبان‌های استفاده‌شده:

1. **پایتون (Python):** به دلیل وجود کتابخانه‌های غنی برای تحلیل متن و پیاده‌سازی الگوریتم‌های یادگیری ماشین.
2. **XGBoost:** یکی از الگوریتم‌های قدرتمند یادگیری ماشین برای مسائل دسته‌بندی که برای پیش‌بینی تقلب در این پروژه به کار گرفته شده است.
3. **کتابخانه‌های تحلیل کد:** برای پردازش کدها و استخراج ویژگی‌هایی مانند تعداد توابع، پیچیدگی سایکلو ماتیک، و تعداد متغیرها.

کتابخانه‌ها:

- **pandas** برای خواندن و پردازش داده‌ها و استخراج اطلاعات مورد نیاز.
- **scikit-learn** برای پیش‌پردازش داده‌ها، تقسیم‌بندی مجموعه داده‌ها و پیاده‌سازی الگوریتم‌های یادگیری ماشین.
- **joblib** برای ذخیره‌سازی و بازیابی مدل‌های یادگیری ماشین و مقیاس‌سنجی.
- **AST (Abstract Syntax Tree)** برای تحلیل ساختار نحوی کدها و مقایسه ساختارهای برنامه‌نویسی.
- **Levenshtein** برای محاسبه شباهت متنی بین دو رشته و مقایسه کدهای برنامه‌نویسی.
- **XGBoost** به‌عنوان الگوریتم یادگیری ماشین که بهینه‌سازی شده و دقت بالایی در مسائل دسته‌بندی دارد.

تکنولوژی‌های پیاده‌سازی:

- **Jupyter Notebooks** برای توسعه و آزمایش الگوریتم‌ها و مشاهده خروجی‌ها به صورت تعاملی.
 - **Git** برای مدیریت نسخه‌های مختلف پروژه و اشتراک‌گذاری کد با اعضای تیم.
 - کتابخانه‌های استاندارد **Python** نظیر **os** و **warnings** برای مدیریت فایل‌ها و گزارش‌های خطا.
- استفاده از این ابزارها و تکنولوژی‌ها باعث شد تا سیستم طراحی شده، هم از نظر کارایی و هم از نظر دقت در تشخیص شباهت‌ها و تقلب‌ها عملکرد خوبی ارائه دهد.

ساختار پروژه

پروژه‌ی ما شامل چندین دایرکتوری و فایل است که هر کدام مسئول بخش‌های مختلف سیستم تشخیص تقلب هستند. در ادامه توضیح مختصری از هر کدام ارائه شده است:

- **idea**: این پوشه شامل فایل‌های پیکربندی پروژه است که توسط محیط توسعه‌ی **PyCharm** ایجاد شده است. این فایل‌ها به مدیریت پروژه و تنظیمات محیط کاری کمک می‌کنند.
- **algorithms**: در این دایرکتوری، الگوریتم‌های مختلفی که برای تشخیص تقلب استفاده می‌شوند، قرار دارند. این الگوریتم‌ها به تحلیل کدها و تعیین میزان شباهت میان آن‌ها می‌پردازند.

- **DataSet** : این پوشه شامل داده‌هایی است که برای تست و آموزش سیستم تشخیص تقلب استفاده می‌شوند. فایل‌های کدهای دانشجویان که برای مقایسه و تحلیل به کار می‌روند در اینجا قرار دارند.
 - **Homework's** : این دایرکتوری شامل تکالیف و کدهای دانشجویان است. سیستم تشخیص تقلب از این کدها برای شناسایی موارد مشکوک به تقلب استفاده می‌کند.
 - **Outputs** : در این پوشه نتایج و خروجی‌های حاصل از اجرای سیستم تشخیص تقلب ذخیره می‌شود. این خروجی‌ها شامل گزارش‌های مربوط به شباهت کدها و تشخیص تقلب می‌باشند.
 - **Utils** : این پوشه شامل ابزارهای کمکی و توابع عمومی است که در بخش‌های مختلف پروژه مورد استفاده قرار می‌گیرند.
 - **main.py** : فایل اصلی پروژه که اجرای کلی سیستم از اینجا آغاز می‌شود. این فایل مسئول مدیریت فرایندهای مختلف و اجرای الگوریتم‌ها برای تشخیص تقلب است.
-

پوشه algorithms

این دایرکتوری شامل فایل‌ها و الگوریتم‌های اصلی پروژه است که برای تشخیص تقلب و مقایسه کدها به کار می‌روند. هر فایل نقش مهمی در سیستم ایفا می‌کند و به تحلیل و ارزیابی شباهت کدهای مختلف کمک می‌کند.

- **ML** : این پوشه شامل مدل‌های یادگیری ماشین (Machine Learning) است که برای پیش‌بینی و تشخیص موارد مشکوک به تقلب به کار می‌روند.
 - **دایرکتوری ML**
 - این دایرکتوری بخشی از پروژه است که به مدل‌های یادگیری ماشین و استخراج ویژگی‌های مرتبط با تشخیص تقلب اختصاص دارد. فایل‌ها و مدل‌های ذخیره شده در اینجا برای پیش‌بینی و تحلیل شباهت کدهای دانشجویان مورد استفاده قرار می‌گیرند.
 - **PLOT** : این پوشه شامل نمودارها و نتایج بصری‌سازی شده باشد که برای ارزیابی عملکرد مدل‌های یادگیری ماشین و تحلیل داده‌ها به کار می‌رود.
 - **cheating_detector_model.pkl** : این فایل مدل آموزش‌دیده یادگیری ماشین است که برای تشخیص تقلب در کدهای دانشجویان استفاده می‌شود. این مدل با داده‌های مختلف آموزش دیده و در نهایت در قالب فایل pkl ذخیره شده است تا در سیستم مورد استفاده قرار گیرد.
 - **dump_model.py** : این فایل اسکریپتی است که برای ذخیره و بارگذاری مدل‌های یادگیری ماشین (مانند cheating_detector_model.pkl) استفاده می‌شود. این اسکریپت به مدیریت مدل‌های مختلف در سیستم کمک می‌کند.
 - **extract_features.py** : این فایل وظیفه‌ی استخراج ویژگی‌های مختلف از کدها را بر عهده دارد. ویژگی‌هایی مانند شباهت AST، شباهت توکن‌ها، و سایر متریک‌ها از طریق این اسکریپت محاسبه و برای آموزش مدل یا استفاده در تحلیل‌ها به کار گرفته می‌شوند.

○ **scaler.pkl**: این فایل شامل مدل مقیاس‌کننده (Scaler) است که برای نرمال‌سازی داده‌ها به کار می‌رود. قبل از آموزش مدل‌های یادگیری ماشین، داده‌ها به مقیاس مشابهی تبدیل می‌شوند تا عملکرد بهتری داشته باشند و این فایل همان مقیاس‌کننده‌ی استفاده شده در فرآیند آموزش است.

- **ast_comparator.py**: این فایل وظیفه‌ی مقایسه درخت نحوی انتزاعی (AST) کدها را بر عهده دارد. یک ساختار داده‌ای است که نشان‌دهنده‌ی نحوه‌ی سازمان‌دهی کدهاست و این فایل شباهت میان آن‌ها را محاسبه می‌کند.
- **block_permutation_detector.py**: این فایل مسئول تشخیص جابه‌جایی بلوک‌های کد است. به این معنی که اگر دانشجویان بلوک‌های کد خود را جابه‌جا کرده باشند، این الگوریتم قادر به تشخیص آن خواهد بود.
- **cheating_detector.py**: این فایل یکی از مهم‌ترین فایل‌هاست و الگوریتم اصلی تشخیص تقلب را پیاده‌سازی می‌کند. این الگوریتم با استفاده از ترکیب چندین تکنیک، به شناسایی شباهت‌های مشکوک در کدها می‌پردازد.
- **code_comparison_dialog.py**: این فایل یک دیالوگ یا واسط کاربری برای مقایسه‌ی کدها فراهم می‌کند و به کاربر امکان می‌دهد نتایج مقایسه را مشاهده کند.
- **extra_features.py**: این فایل شامل توابع اضافی برای محاسبه‌ی ویژگی‌های خاصی از کدها مانند نسبت کامنت‌ها، پیچیدگی حلقوی (Cyclomatic Complexity)، و تعداد توابع است که به تشخیص بهتر تقلب کمک می‌کند.
- **levenshtein.py**: این فایل پیاده‌سازی الگوریتم فاصله‌ی لوئش‌تین است که میزان تفاوت بین دو رشته (یا کد) را محاسبه می‌کند. هر چه این فاصله کمتر باشد، شباهت بین کدها بیشتر است.
- **similarity_detector.py**: این فایل وظیفه‌ی تشخیص شباهت بین کدها را با استفاده از روش‌های مختلف، از جمله شباهت توکن‌ها و ساختارهای کد، بر عهده دارد.
- **tokenizer.py**: این فایل وظیفه‌ی تجزیه و تحلیل کدها به توکن‌ها (واحد‌های کوچک‌تر) را بر عهده دارد. توکن‌ها عناصر بنیادی کد هستند و این فایل به شناسایی آن‌ها کمک می‌کند تا شباهت‌ها و تفاوت‌های ساختاری بین کدها تشخیص داده شود.

پوشه DataSet

پوشه DataSet شامل فایل‌ها و داده‌های مرتبط با مدل یادگیری ماشین می‌باشد:

- **cheating_dataset.csv**: این فایل باید توسط مدیر پروژه تکمیل شود و اطلاعات پایه‌ی مجموعه داده‌های تقلب را در بر می‌گیرد.
- **cheating_features_dataset.csv**: این فایل خروجی مدل یادگیری ماشین است و شامل ویژگی‌های استخراج‌شده از کدهای دانشجویان به همراه برجسب‌های تقلب می‌باشد.
- **Submission 1 تا 174**: این پوشه‌ها شامل داده‌های ارسالی دانشجویان است که مدل یادگیری ماشین بر روی آن‌ها آموزش می‌بیند. مدل از این ارسال‌ها به همراه فایل `cheating_features_dataset.csv` (که

برچسب‌های مربوط به تقلب در آن است) استفاده کرده و در نهایت فایل `cheating_features.csv` را تولید می‌کند که ویژگی‌های نهایی را در اختیار قرار می‌دهد.

پوشه Home Work

محل قرار گیری فایل های تمرین ها است. در این پروژه برای تست این پوشه قرار داده شده است و این پوشه میتواند هر جایی باشد چون کاربر خود میتواند محل قرار گیری این پوشه را مشخص کند تنها شرطی که وجود دارد این که تمام تکالیف باید در یک پوشه باشند

پوشه OutPut

این پوشه هم مانند پوشه قبل برای مدیریت بهتر قرار داده شده که فایل های اکسل که خروجی هستند را در آن قرار بدهیم البته مانند پوشه قبلی میتوان این فایل های اکسل را به هر مسیر دلخواه ذخیره سازی کرد

پوشه Utils

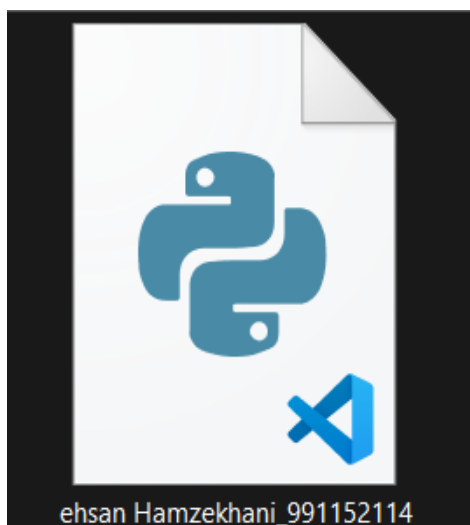
پوشه `Utils` شامل ابزارهای کمکی است که در بخش‌های مختلف پروژه مورد استفاده قرار می‌گیرند:

- **excel_exporter.py**: این فایل شامل توابعی است که برای خروجی گرفتن گزارشات به صورت فایل اکسل استفاده می‌شوند. به خصوص در پروژه‌ی شناسایی تقلب، از این فایل برای صدور گزارش‌های مختلف در قالب Excel استفاده می‌شود.
- **file_reader.py**: این فایل حاوی توابعی برای خواندن فایل‌ها است که در بخش‌های مختلف برنامه از آن برای پردازش و خواندن ورودی‌های فایل‌ها استفاده می‌شود.
- **pycache**: پوشه‌ی کش مربوط به فایل‌های کامپایل شده پایتون برای افزایش سرعت اجرای برنامه است.

توضیحات نحوه عملکرد سیستم

این سیستم کشف تقلب با دو مدل وابسته به هوش مصنوعی و مدل دیگر به صورت ساختاری تکالیف ارسالی را مورد بررسی قرار میدهد که تلفیق این دو موجب افزایش دقت و قدرت این سیستم شده که بتواند شباهت و تقلب های بین دانشجویان را به طور کامل شناسایی کند

نحوه عملکرد این سیستم به این صورت است که استاد(فرد مصحح) تکالیف دانشجوییان که به صورت کد هست را دریافت میکنند و در یک پوشه قرار میدهند البته لازم به ذکر است که فرمت تحویلی تکالیف باید به صورت عکس زیر باشد :



۱-۳- نحوه نام گذاری فایل

فایل ارسالی از سمت دانشجو باید به صورت زیر باشد یعنی

یک فایل حاوی کد به زبان پایتون یا سی پلاس که نام گذاری فایل

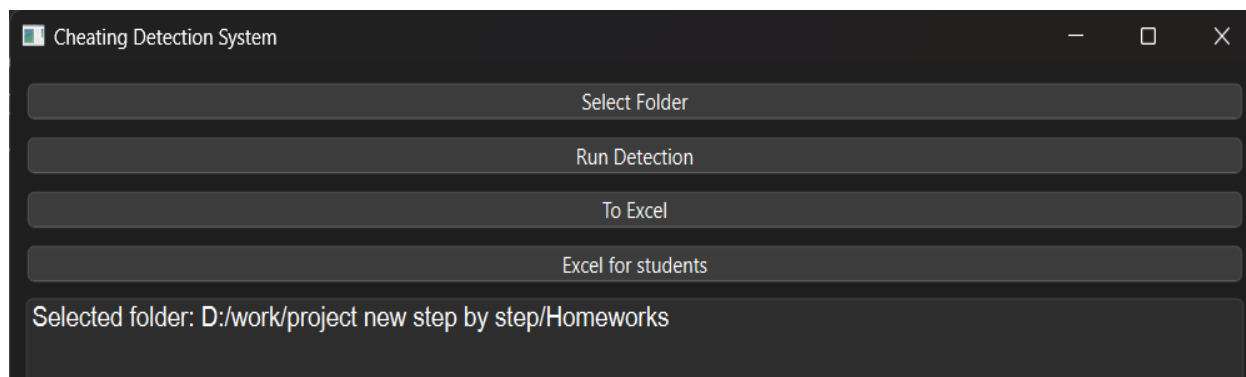
به صورت روبه رو است:

نام کامل - شماره دانشجویی. py or cpp

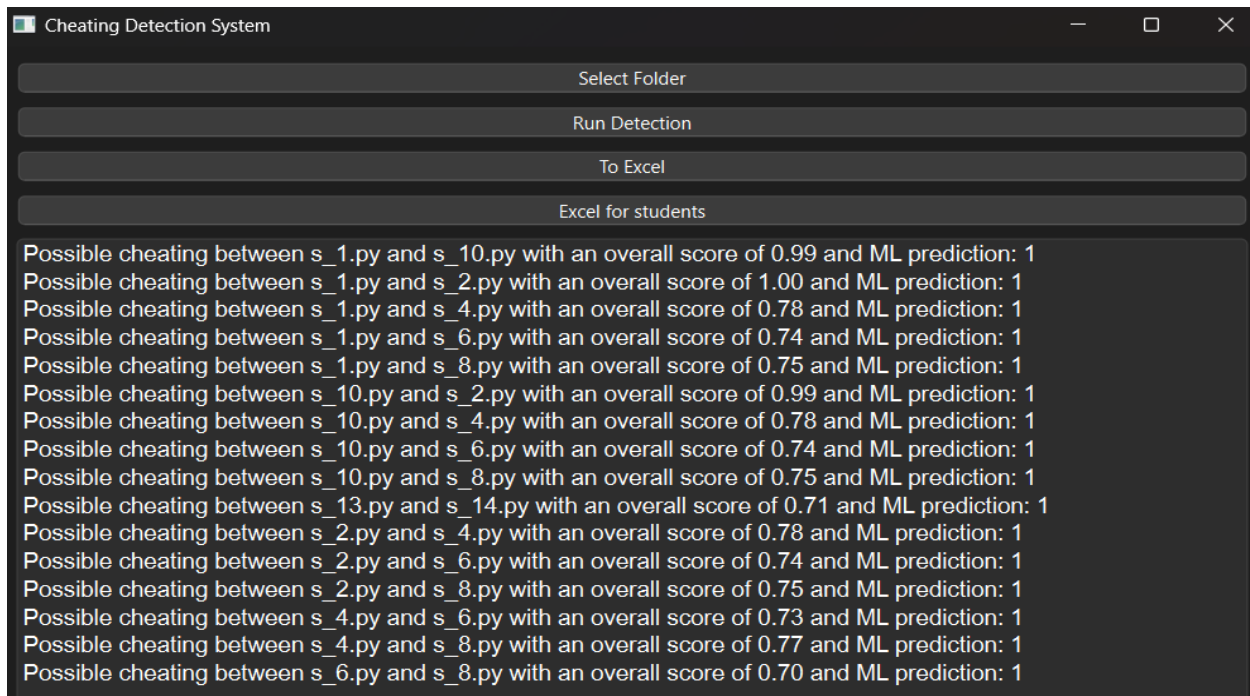
در قسمت های بعدی این نام گذاری باید حتما به صورت روبه رو

باشد زیر در قسمت های بعدی این نام مورد استفاده قرار میگیرد.

در مرحله بعد که سیستم پوشه تکالیف را شناسایی کرد میتوان از منو طراحی شده گزینه run detection را انتخاب کرد که سیستم شروع به کار کند که دو مدل را روی تمام تکالیف به صورت دو به دو تست میکند تا تمام شباهت و تقلب های ممکن را شناسایی کند. که خروجی به شکل روبه رو به کاربر یا فرد مصحح نشان داده میشود.



۲-۳- انتخاب پوشه تکالیف



۳-۳- اجرای مدل ها جهت شناسایی تقلب

در این بخش مدل بین تمامی فایل ها به صورت دو به دو اجرا میشود که تمامی ترکیب های ممکن جهت پیدا کردن تقلب را پیدا کند. همانطور که در شکل مشخص است در خروجی بین هر دو دانشجو یک عدد برای مدل ساختاری بین ۰ و ۱۰۰ که نمایانگر درصد شباهت است و عدد دوم نشانگر تشخیص مدل هوش مصنوعی به صورت ۰ و ۱ است. در هر خط از خروجی این مقایسه بین دو دانشجوی فرضی با نام s و شماره دانشجویی های ۱ تا ۱۴.

در صورت اشتباه مدل ها یا شک کردن فرد مصحح، آن شخص میتواند با انتخاب کردن هر خط هر دو کد را برای خود ببیند که بتواند ملاحظات خودش را از آن تشخیص مدل بنویسد.


```
Code Comparison

s_1.py
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)

print(factorial(5))

s_10.py
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(9))
```

۴-۳- مقایسه دو به کد ها باهم

در مرحله بعد دو عدد دکمه جهت تبدیل تمام داده های به دست آمده از به صورت اکسل هستیم چون در سیستم آموزش یکی از روش های انتقال نمرات بین استاد و دانشجو ها فایل اکسل هست که قابلیت های بالایی دارد از این مدل روش انتقال استفاده کرده ایم. در ادامه خروجی اکسل هر دو فایل اکسل را مشاهده میکنیم.

Report saved to: D:/work/project new step by step/Outputs/test.xlsx
Student report saved to: D:/work/project new step by step/Outputs/student.xlsx

۵-۳- پیام عملیات موفقیت امیز برای تولید هر دو فایل اکسل

Name	ID	Actual Number	Cheat (%)	Final Grade
s	1		100	0
s	2		100	0
s	3		0	
s	4		78	
s	5		0	
s	6		74	
s	7		0	
s	8		77	
s	9		0	
s	10		99	
s	11		0	
s	12		0	
s	13		71	
s	14		71	

Detailed Cheating Report

Possible cheating between s_1.py and s_10.py with an overall score of 0.99 and ML prediction: 1

Possible cheating between s_1.py and s_2.py with an overall score of 1.00 and ML prediction: 1

Possible cheating between s_1.py and s_4.py with an overall score of 0.78 and ML prediction: 1

Possible cheating between s_1.py and s_6.py with an overall score of 0.74 and ML prediction: 1

Possible cheating between s_1.py and s_8.py with an overall score of 0.75 and ML prediction: 1

Possible cheating between s_10.py and s_2.py with an overall score of 0.99 and ML prediction: 1

Possible cheating between s_10.py and s_4.py with an overall score of 0.78 and ML prediction: 1

Possible cheating between s_10.py and s_6.py with an overall score of 0.74 and ML prediction: 1

Possible cheating between s_10.py and s_8.py with an overall score of 0.75 and ML prediction: 1

Possible cheating between s_13.py and s_14.py with an overall score of 0.71 and ML prediction: 1

Possible cheating between s_2.py and s_4.py with an overall score of 0.78 and ML prediction: 1

Possible cheating between s_2.py and s_6.py with an overall score of 0.74 and ML prediction: 1

Possible cheating between s_2.py and s_8.py with an overall score of 0.75 and ML prediction: 1

Possible cheating between s_4.py and s_6.py with an overall score of 0.73 and ML prediction: 1

Possible cheating between s_4.py and s_8.py with an overall score of 0.77 and ML prediction: 1

Possible cheating between s_6.py and s_8.py with an overall score of 0.70 and ML prediction: 1

End of Detailed Cheating Report

۶-۳- خروجی دکمه TO EXCEL

این قسمت مختص به استاد یا فرد مصحح است که با این اکسل میتواند به تمام جزئیات تمامی تکالیف

دست پیدا کند در یکی از ستون ها نام کامل دانشجو و در ستون بعد شماره دانشجویی (برای دید بهتر در این

قسمت نام تمام دانشجویان به s و تمام شماره های دانشجویی به ترتیب ۱ تا ۱۴ هستند). در این قسمت یک

ستون actual number داریم که توسط خود استاد بعد از تصحیح وارد میشود. اگر توجه کرده باشید در عکس

بالا اگر یکی از ستون های قسمت cheat مساوی با عدد ۱۰۰ باشد سیستم به صورت اتوماتیک نمره نهایی دانشجو را صفر قرار میدهد و به این معنا است که این جفت دانشجو یک تکلیف را دوبار(به صورت کاملاً کپی از هم) تحویل داده اند که بالا ترین درصد تقلب را نشان میدهد.

Name	ID	Final Grade
s	1	0
s	2	0
s	3	
s	4	
s	5	
s	6	
s	7	
s	8	
s	9	
s	10	
s	11	
s	12	
s	13	
s	14	

در این قسمت با استفاده از دکمه excel for students این فایل اکسل ساخته میشود که محتویات آن به شکل رو به رو و مانند فایل قبلی است ولی با این تفاوت که آن جزییات قبل را ندارد و مختص به این کار است که از طرف فرد مصحح به دانشجویان ارسال شود.

نحوه اجرای پروژه

برای ران کردن پروژه در ابتدا باید مدل هوش مصنوعی که یک نوع از روش های یادگیری ماشین است باید اجرا و نتایج آن ذخیر شود در قسمت های بالا توضیح داده شده در مورد فایل با فرمت pkl که همان نتایج این مدل را در خود نگه میدارد.

در ابتدا باید فایل extract features ران شود تا از روی دیتاست و یک فایل csv فایل جدیدی به وجود می آید که فرمت آن به صورت زیر است :

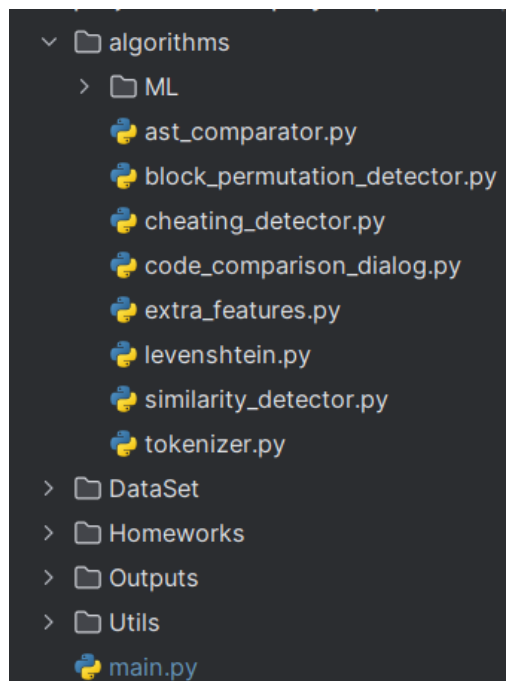
```
AST Similarity,Token Similarity,Levenshtein Similarity,Length File 1,Length File 2,Function Count File 1,Function Co
1.0,1.0,1.0,2,2,1,1,4,4,0.0,0.0,1,1,1
0.0,0.9411764705882353,0.7837837837837838,2,2,1,1,4,4,0.0,0.0,1,1,1
0.0,0.47619047619047616,0.5576923076923077,2,3,1,1,4,5,0.0,0.0,1,1,1
0.0,0.9411764705882353,0.7567567567567568,2,2,1,1,4,4,0.0,0.0,1,1,0
0.0,0.9411764705882353,0.7837837837837838,2,2,1,1,4,4,0.0,0.0,1,1,0
0.0,0.47619047619047616,0.5576923076923077,2,3,1,1,4,5,0.0,0.0,1,1,1
0.0,0.9411764705882353,0.7567567567567568,2,2,1,1,4,4,0.0,0.0,1,1,0
0.0,0.47619047619047616,0.40384615384615385,2,3,1,1,4,5,0.0,0.0,1,1,0
0.0,0.9411764705882353,0.8108108108108107,2,2,1,1,4,4,0.0,0.0,1,1,0
```

۸-۳- تصویری از فایل cheating features dataset

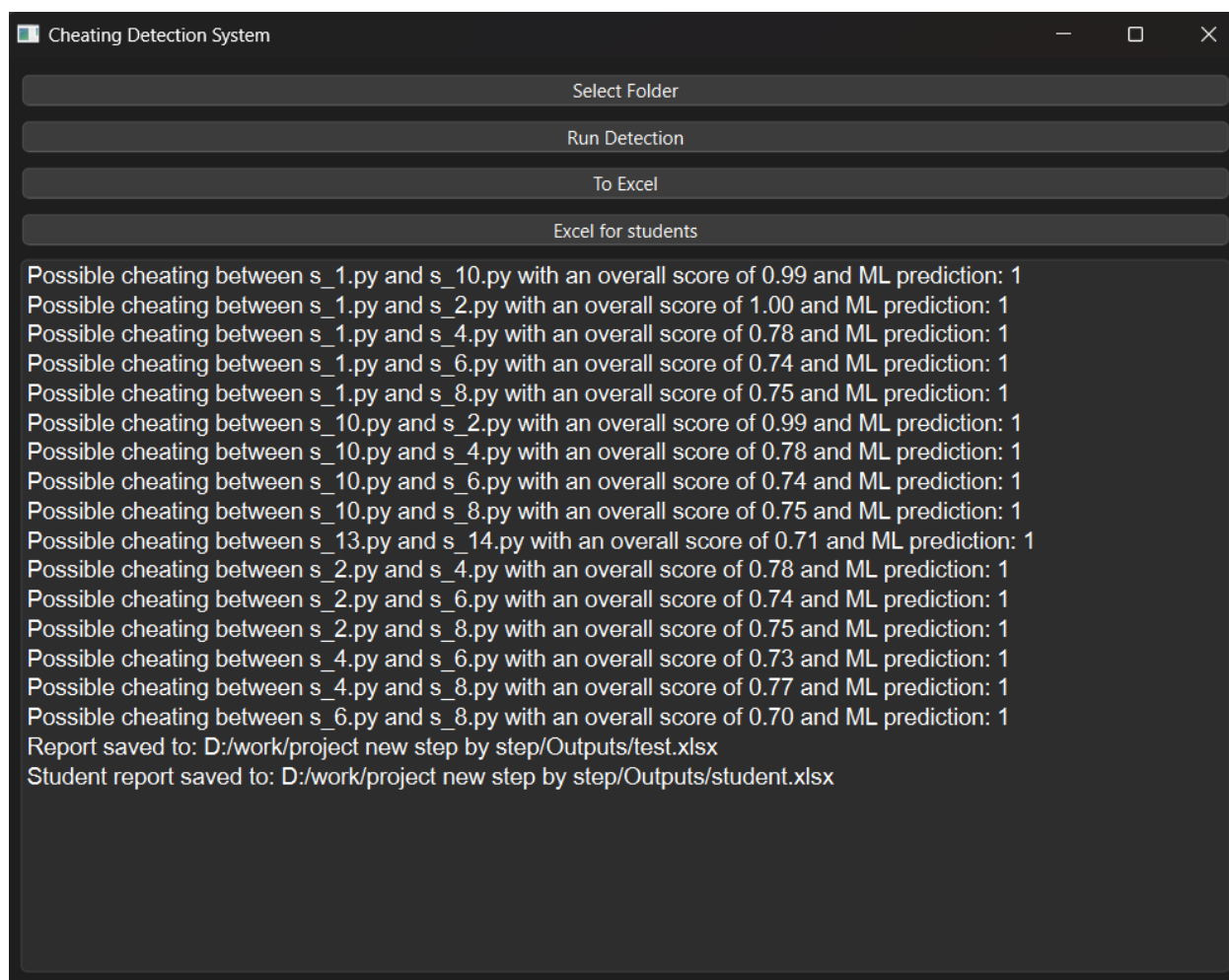
بعد از این که ویژگی های مدل به طور کامل به دست امد نوبت میرسد به تبدیل و ران کردن مدل جهت این که در سیستم اصلی ما مورد بررسی قرار بگیرد، در واقع در این قسمت است که فایل های pkl به وجود می آیند.

بعد این مرحله سیستم ما آماده است که ران بشود که یعنی میتوان فایل main را اجرا کرد که منوی برنامه باز بشود که توضیحات منو در قسمت های بالایی به طور کامل پرداخته شده است.

تصاویری از محیط پروژه



۹-۳- ساختار کلی پروژه



۱۰-۳- منوی اصلی برنامه

نمودار ها

در یکی از مدل های این سیستم که به بررسی ساختاری کد ها برای شناسایی تقلب میپردازد با استفاده از چند روش و الگوریتم این مقدار پیش بینی را تقسیم کرده تا مدلی دقیق تر داشته باشیم .

```

overall_score = (
    0.1 * text_sim_score +
    0.2 * ast_sim_score +
    0.5 * token_sim_score +
    0.2 * lev_sim_score
)

if overall_score > 0.60 and ml_prediction == 1:
    enhanced_results.append((file1, file2, overall_score, ml_prediction))

```

۱۱-۳- درصد بندی در مدل ساختاری

همانطور که ملاحظه میشود این مدل ساختاری به چند بخش با ضریب های مختلف تقسیم بندی شده

است که دقت مدل ما را افزایش میدهد همچنین در ادامه داریم که اگر درصد بدست آمده بالای ۶۰ و پیش

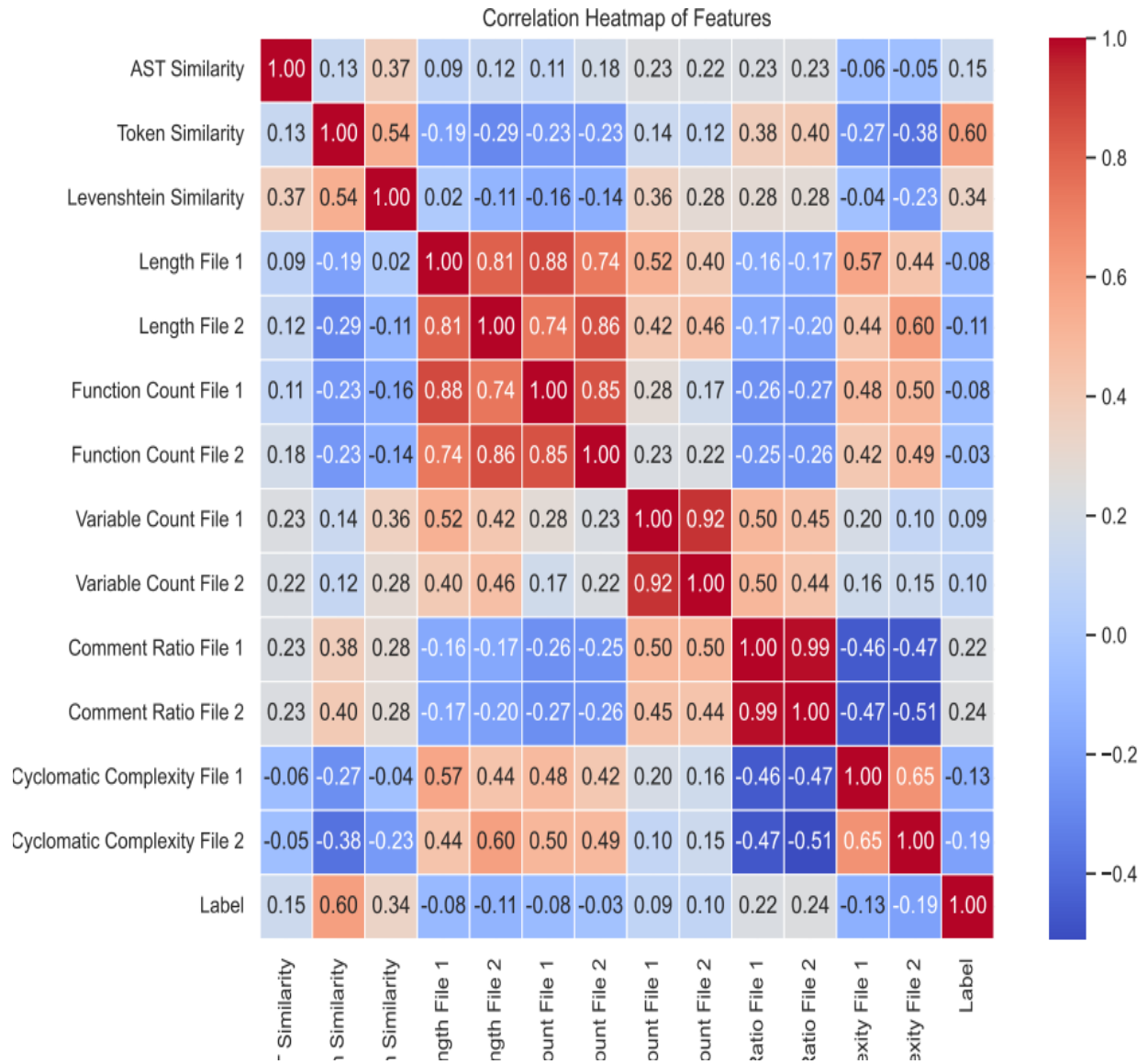
بینی مدل یادگیری ماشین مثبت باشد انرا به صورت تقلب برای ما گزارش کند.

حال برای این که بفهمیم بهترین ترکیب برای پخش کردن ضریب های کدام است باید از نمودار ها بهره

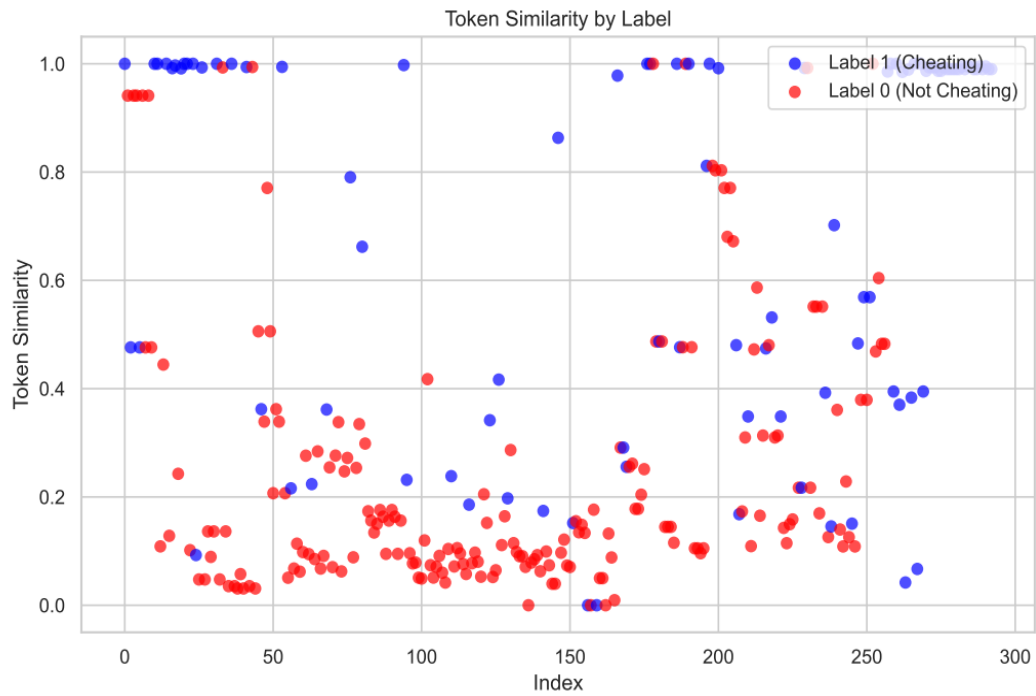
بگیریم.

نمودار های در فایل ml- plots قرار دارند که در ان یک فایل ipynb و بقیه تمامی نمودار های بدست آمده

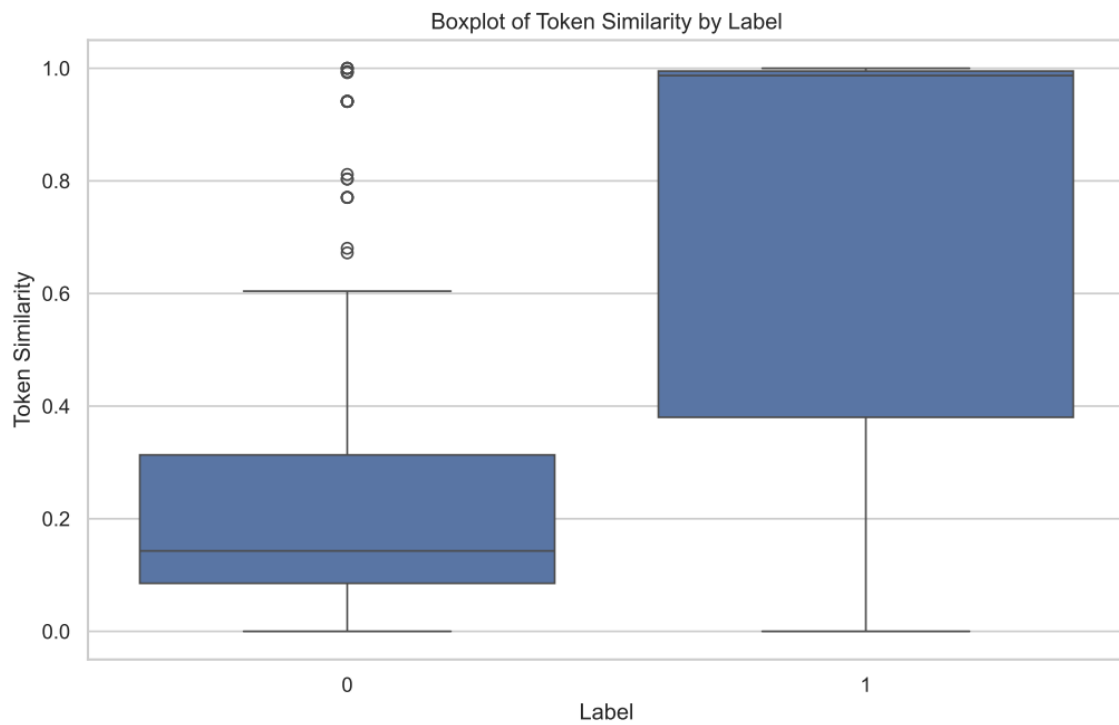
از این دیتا ها هستن که ما ببینیم کدام ویژگی بیشترین تاثیر را در مدل ما دارد.



۱۲-۳- تصویر heatmap



token_similarity_plot - 3-13



Token Similarity_boxplot - 3-14

مطالعه منابع

مقاله ۱: سیستمی برای تشخیص سرقت ادبی نرم افزار

- نویسندگان M. Joy، N. Griffiths
- مجله Computers & Education، 2008
- خلاصه: این مقاله به بررسی یک سیستم جامع برای شناسایی سرقت ادبی در تکالیف برنامه نویسی می پردازد. این سیستم بر پایه روش های مختلفی برای مقایسه کدها طراحی شده است، از جمله تکنیک های توکنیزاسیون و تحلیل ساختاری. یکی از اهداف این سیستم، خودکارسازی فرایند تشخیص سرقت ادبی است که به معلمان اجازه می دهد به سرعت تکالیف دانشجویان را تحلیل کنند و شباهت های مشکوک را کشف کنند. این سیستم به جای تمرکز تنها بر شباهت های سطحی کدها، ساختار و نحوه نگارش آن ها را نیز در نظر می گیرد. علاوه بر این، مقاله به چالش های فنی مرتبط با این حوزه اشاره می کند، از جمله مشکلات مرتبط با تغییرات جزئی در کدها که ممکن است منجر به پنهان کردن سرقت ادبی شود. به طور کلی، این مطالعه به ارائه الگوریتم های پیشرفته ای می پردازد که می توانند در محیط های آموزشی برای تشخیص خودکار سرقت ادبی استفاده شوند.

مقاله ۲: تشخیص سرقت ادبی با استفاده از الگوریتم های یادگیری ماشین

- نویسندگان T. Fatima، K. Raza
- کنفرانس 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS)
- خلاصه: در این مقاله، استفاده از الگوریتم های یادگیری ماشین برای تشخیص سرقت ادبی در کدهای برنامه نویسی بررسی شده است. نویسندگان با استفاده از طبقه بندی های مختلف مانند درخت تصمیم، ماشین های بردار پشتیبان (SVM) و سایر الگوریتم های یادگیری ماشین به شناسایی الگوهای مشترک در کدها پرداخته اند. در این مطالعه، ابتدا ویژگی های مختلفی از کدهای برنامه نویسی استخراج شده و سپس این ویژگی ها به عنوان ورودی برای الگوریتم های یادگیری ماشین استفاده شده اند. هدف اصلی این پژوهش، ارائه روشی دقیق تر و کارآمدتر برای تشخیص سرقت ادبی با استفاده از الگوریتم های هوش مصنوعی است. نویسندگان همچنین به مقایسه عملکرد الگوریتم های مختلف پرداخته و تأثیرات استفاده از هر یک را بر دقت تشخیص سرقت ادبی بررسی کرده اند. این مقاله تأکید می کند که استفاده از تکنیک های یادگیری ماشین می تواند دقت و سرعت تشخیص را بهبود بخشد، به ویژه در مواردی که کدها به صورت هوشمندانه تغییر کرده اند تا شناسایی نشوند.

مقاله ۳: تشخیص سرقت ادبی با استفاده از الگوریتم‌های شباهت متنی

- نویسندگان B. Zaka, F. Kappe, H. Maurer
- مجله Journal of Universal Computer Science, 2006
- خلاصه: این مقاله به بررسی الگوریتم‌های مختلفی برای تشخیص سرقت ادبی در متون و کدهای برنامه‌نویسی پرداخته است که یکی از مهم‌ترین آن‌ها، فاصله لوناشتاین (Levenshtein distance) است. فاصله لوناشتاین به عنوان معیاری برای اندازه‌گیری تفاوت بین دو رشته متنی معرفی می‌شود و با توجه به تعداد تغییرات لازم برای تبدیل یک رشته به رشته دیگر، فاصله بین آن‌ها را محاسبه می‌کند. این روش به دلیل سادگی و کارایی بالای خود، به طور گسترده در تشخیص سرقت ادبی به کار گرفته می‌شود. در این مقاله، کاربردهای مختلف این الگوریتم در حوزه تشخیص سرقت ادبی مورد بررسی قرار گرفته و با سایر الگوریتم‌های شباهت متنی مقایسه شده است. نویسندگان همچنین نشان داده‌اند که چگونه فاصله لوناشتاین می‌تواند در تشخیص تقلب در برنامه‌نویسی و سایر انواع اسناد به کار گرفته شود. این مقاله تأکید می‌کند که استفاده از این الگوریتم در کنار سایر روش‌ها می‌تواند دقت تشخیص سرقت ادبی را افزایش دهد.

مقاله ۴: تحلیل شباهت ساختاری برای تشخیص سرقت ادبی

- نویسندگان A. Aiken, J. Schleimer
- کنفرانس ACM SIGMOD International Conference on Management of Data
- خلاصه: این مقاله به بررسی یک روش پیشرفته برای تشخیص شباهت در کدهای برنامه‌نویسی بر اساس تجزیه درخت نحوی مجرد (AST) می‌پردازد. تحلیل AST یکی از ابزارهای قدرتمند در بررسی ساختار کدهای برنامه‌نویسی است. در این روش، کدها به درخت‌های نحوی تجزیه می‌شوند که هر گره در این درخت نمایانگر یک مؤلفه از ساختار برنامه است. این مقاله نشان می‌دهد که چگونه با استفاده از مقایسه درخت‌های نحوی، می‌توان شباهت‌های ساختاری بین کدهای برنامه‌نویسی را شناسایی کرد. تحلیل AST به‌ویژه در مواردی که دانشجویان تغییرات سطحی در کد اعمال کرده‌اند تا تقلب پنهان شود، مؤثر است. نویسندگان به بررسی کارآمدی این روش در مقایسه با سایر روش‌ها پرداخته‌اند و نتیجه گرفته‌اند که AST می‌تواند به‌طور قابل توجهی دقت تشخیص سرقت ادبی را افزایش دهد.

مقاله ۵: تشخیص سرقت ادبی در کدهای منبع با استفاده از توکنیزاسیون

- نویسندگان S. Robinson, E. Faidhi
- مجله IEEE Transactions on Education, 1987
- خلاصه: این مقاله یکی از اولین مطالعات جامع در زمینه تشخیص سرقت ادبی در کدهای برنامه‌نویسی با استفاده از تکنیک توکنیزاسیون است. در این روش، کدها به توکن‌های کوچک‌تری مانند کلمات کلیدی، عملگرها و شناسه‌ها تبدیل

می‌شوند. سپس این توکن‌ها با استفاده از الگوریتم‌های مقایسه‌گر بررسی می‌شوند تا شباهت‌های موجود بین کدها شناسایی شود. نویسندگان نشان داده‌اند که توکنیزاسیون یکی از پایه‌های اصلی بسیاری از روش‌های مدرن تشخیص سرقت ادبی است. این روش به دلیل سادگی و سرعت بالای خود به طور گسترده در سیستم‌های تشخیص سرقت ادبی استفاده می‌شود. همچنین مقاله به چالش‌های مربوط به این روش مانند تأثیر تغییرات کوچک در کدها بر روی دقت تشخیص اشاره می‌کند.

مقاله ۶: مروری جامع بر تشخیص سرقت ادبی در نرم‌افزار

- نویسندگان: B. Sim, A. Aiken
- مجله: ACM Computing Surveys, 2006
- خلاصه: این مقاله به بررسی جامع سیستم‌های مختلف تشخیص سرقت ادبی در نرم‌افزار می‌پردازد. نویسندگان روش‌های مختلفی را که برای تشخیص تقلب در برنامه‌نویسی به کار می‌روند، از جمله روش‌های مبتنی بر توکنیزاسیون، تحلیل ساختاری و معیارهای شباهت، به تفصیل بررسی کرده‌اند. همچنین مقاله به مقایسه مزایا و معایب هر یک از این روش‌ها پرداخته و راهکارهایی برای بهبود آن‌ها ارائه داده است. این مرور سیستماتیک به خوانندگان یک دید کلی از روند پیشرفت تکنولوژی‌های تشخیص سرقت ادبی در طی سال‌های اخیر ارائه می‌دهد. هدف اصلی این مقاله، ارائه رویکردی جامع و سیستماتیک برای تشخیص سرقت ادبی در نرم‌افزار است.

مقاله ۷: استفاده از یادگیری ماشین در تشخیص خودکار سرقت ادبی

- نویسندگان: L. Figueras, J. Roca
- کنفرانس: International Conference on Artificial Intelligence, 2018
- خلاصه: این مقاله به بررسی کاربرد الگوریتم‌های یادگیری ماشین در تشخیص خودکار سرقت ادبی می‌پردازد. نویسندگان از تکنیک‌هایی مانند درخت تصمیم، جنگل تصادفی و ماشین بردار پشتیبان (SVM) برای تحلیل کدهای برنامه‌نویسی استفاده کرده‌اند. در این مطالعه، ابتدا ویژگی‌های مختلفی از کدها استخراج و سپس از این ویژگی‌ها برای آموزش مدل‌های یادگیری ماشین استفاده شده است. نتایج نشان می‌دهد که استفاده از الگوریتم‌های یادگیری ماشین می‌تواند دقت و سرعت تشخیص سرقت ادبی را بهبود بخشد. این مقاله همچنین به چالش‌های مرتبط با انتخاب ویژگی‌های مناسب برای مدل‌های یادگیری ماشین و تأثیر آن‌ها بر دقت سیستم اشاره می‌کند.

- نویسندگان: S. Burrows, A. Tahaghoghi
- مجله: Journal of Software: Evolution and Process, 2007
- خلاصه: این مقاله به بررسی استفاده از معیارهای ساختاری برای تشخیص شباهت در کدهای برنامه‌نویسی می‌پردازد. نویسندگان بر این باورند که معیارهای ساختاری مانند نحوه تعریف کلاس‌ها، توابع و متغیرها می‌توانند به شناسایی شباهت‌های پنهان بین کدها کمک کنند. این روش به خصوص زمانی مؤثر است که کدها با تغییرات سطحی اصلاح شده‌اند تا تقلب پنهان شود. مقاله نشان می‌دهد که با استفاده از این معیارها، می‌توان دقت تشخیص سرقت ادبی را بهبود داد و حتی مواردی را که با تغییرات جزئی در کد پوشانده شده‌اند، شناسایی کرد.

خطاهای احتمالی

این برنامه به دلیل تعداد مقایسه های زیاد بین هر فایل و داشتن تعداد زیادی ویژگی میتواند در برخی موارد دچار مشکل رم بشود که تا حدودی این مشکل رفع شده . از ایرادات و مشکلاتی که شاید به آن برخورد کنیم نحوه نام گذاری فایل هست که دقیقاً باید طبق همان فرمت ذکر شده نام گذاری شود.

برای اضافه کردن دیتا جهت بهبود مدل یادگیری ماشین هم باید فایل های کد ها به عنوان دیتا ست در پوشه مورد نظر قرار داده شود و هم این که در فایل cheating dataset هم ذکر بشود که بین آن دو فایل ایا تقلبی رخ داده یا خیر که با صفر و یک مشخص میشود.

مراجع

به عنوان مراجع در مرحله اول میتوان به دروسی که در دوره کارشناسی رشته کامپیوتر اشاره کنیم مانند دروس کامپایلر، نظریه اعداد و ماشین ها، ساختمان داده اشاره کنیم

فایل های کد پروژه به همراه توضیحات را میتوان در ادرس زیر پیدا کرد:

github.com/ehsankhani/cheating_detector

در ادامه به اشاره منابع میپردازیم:

Cheating Detection in Programming Assignments

- **Title:** "A System for Detecting Software Plagiarism"
 - **Author:** M. Joy, N. Griffiths
 - **Journal:** Computers & Education, 2008.
 - **Summary:** This paper discusses a system for detecting plagiarism in programming assignments, comparing various methods like tokenization and structural analysis.

2. Machine Learning for Plagiarism Detection

- **Title:** "Plagiarism Detection using Machine Learning Algorithms"
 - **Author:** T. Fatima, K. Raza
 - **Journal:** 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS)
 - **Summary:** This study explores the use of machine learning techniques to detect plagiarism in code, including feature extraction and the role of classifiers like decision trees and SVM.

3. Levenshtein Distance in Plagiarism Detection

- **Title:** "Plagiarism Detection using Text Similarity Algorithms"
 - **Author:** H. Maurer, F. Kappe, B. Zaka
 - **Journal:** Journal of Universal Computer Science, 2006.
 - **Summary:** This paper examines various text similarity algorithms, including Levenshtein distance, and their effectiveness in detecting plagiarism in code and text documents.

4. Abstract Syntax Tree for Code Similarity Detection

- **Title:** "Structural Similarity Analysis for Plagiarism Detection"
 - **Author:** J. Schleimer, A. Aiken
 - **Conference:** ACM SIGMOD International Conference on Management of Data
 - **Summary:** This paper presents a method for detecting similarity in code based on AST (Abstract Syntax Tree) analysis, showing its efficiency for plagiarism detection.

5. Tokenization in Source Code Plagiarism Detection

- **Title:** "Detecting Plagiarism in Source Code Using Tokenization"
 - **Author:** E. Faidhi, S. Robinson
 - **Journal:** IEEE Transactions on Education, 1987.
 - **Summary:** One of the classic studies on detecting plagiarism in source code using tokenization techniques, which forms the basis of many modern approaches.

6. A Comprehensive Review of Software Plagiarism Detection

- **Title:** "Software Plagiarism Detection: A Systematic Review"
 - **Author:** A. Aiken, B. Sim
 - **Journal:** ACM Computing Surveys, 2006.

- **Summary:** A comprehensive review of various software plagiarism detection systems, including token-based and structural-based methods like AST and similarity metrics.

7. ML Techniques in Plagiarism Detection

- **Title:** "Using Machine Learning in the Automatic Detection of Plagiarism"
 - **Author:** J. Roca, L. Figueras
 - **Conference:** International Conference on Artificial Intelligence, 2018.
 - **Summary:** This paper discusses various machine learning approaches like decision trees, random forests, and support vector machines in the context of plagiarism detection.

8. Code Similarity Detection Using Structural Metrics

- **Title:** "Using Structural Metrics to Detect Similarities in Java Programs"
 - **Author:** S. Burrows, A. Tahaghoghi
 - **Journal:** Journal of Software: Evolution and Process, 2007.
 - **Summary:** A paper that investigates structural code similarity techniques and how they can be used in plagiarism detection systems.

ارائه پیشنهادات

با توجه به این که این پروژه میتواند ابعاد بسیاری را مورد بررسی قرار دهد از این رو فضا برای گسترش این پروژه وجود دارد و در نهایت میتوان یک سیستم کامل برای استاد طراحی کرد که تمام موارد را مورد بررسی قرار دهد به عنوان مثال در مرحله بعد میتوان به آن کامپایلر اضافه کرد که کد سیستم بعد از شناسایی تقلب کدها را ران کرده تا چک بشود که کدها صحت لازم و منطقی را دارند. در نهایت این پروژه کوچک میتواند یک دستیار بزرگ برای استادان دانش که با کمک آنها با دقت و سرعت بالاتر بتوان به بررسی با کیفیت روند آموزش بپردازند.